マクロ計量経済学

第6回　講義ノート
マクロ動学モデルの解法　その6

2025年5月14日

飯星　博邦

outline

1. Why 2nd approximation to policy function is needed
2. Set up of the method
3. 1st approximation to policy function
4. 2nd approximation to policy function
   - Matlab code by Schmitt-Grohe & Uribe (2004, JEDC)
5. Example 1: Neo-classical model
   - Matlab Codes
6. Example 2: two-country RBC model
   - Matlab Codes
7. Example 3: Asset price model
   - Matlab code

Reference

1. Miao (2014) chapter 2, 2-4, and 2-5
2. Schmitt-Grohe and Uribe (2004)
   "Solving dynamic general equilibrium models using a second-order approximation to the policy function," JEDC, 28, 755-775.
3. Coeurdacier, Rey, Winant (2011)
   "The Riskey Steady State," AER, 101:3, 398-401.
4. De Groot (2013)
   "Computing the risky steady state of DSGE models," Economics Letters, 120, 566-569.

マクロ経済の動学モデルの解法

- 解法その1: 確率的ベルマン方程式　Stochastic Bellman Equation
  → 応用範囲が広いが、数値的解法で時間がかかる。
  - 状態変数の動学は、マルコフ決定過程
  - ラグランジュ方程式に置き換えることが可能。

- 解法その2: 合理的期待モデル　Rational Expectation Model
  ⇒　数値的解法での負担が少ない。
  - Uhlig (1997)
  - Blanchard & Kahn (1980)
  - Schmit-Grohe & Uribe (2004)

- 均衡解の動学式はマルコフ決定過程
  - 行列Pは　固有値が1以下で安定

$$x_t = Px_{t-1} + Qz_t, \qquad \text{均衡解}$$

$$y_t = Rx_{t-1} + Sz_t.$$

合理的期待モデル　E_t(f(y_t+1, x_t+1)) に policy function を代入

Many nonlinear rational expectations models can be represented by the following system:

Control variables　State variables

$$E_t\left[f\left(y_{t+1}, y_t, x_{t+1}, x_t\right)\right] = 0, \tag{2.27}$$

where x_t is an $(n_k + n_z) \times 1$ vector of predetermined variables and y_t is an

State variables

$n_y \times 1$ vector of non-predetermined variables in the sense of Blanchard and Kahn (1980). Let $n = n_k + n_z + n_y$ and assume the function $f : \mathbb{R}^{2n} \to \mathbb{R}^n$.

Control variables

The vector x_t consists of n_k endogenous variables and n_z exogenous shocks:

Suppose that the solution to (2.27) takes the following form:

$$y_t = g(x_t, \sigma), \quad x_{t+1} = h(x_t, \sigma) + \eta\sigma\varepsilon_{t+1}, \tag{2.29}$$

Policy function

Dynamics of predetermined variable

where g and h are functions to be determined and η is an $n_x \times n_\varepsilon$ matrix defined by

$$\eta = \begin{bmatrix} 0 \\ \Sigma \end{bmatrix}.$$

合理的期待モデルに1st order approximationを代入

. We totally differentiate (2.27) to obtain the linearized system:

合理的期待モデルの全微分

$$f_{y'}\ E_t dy_{t+1} + f_y\ dy_t + f_{x'}\ E_t dx_{t+1} + f_x\ dx_t = 0,$$

y‘=y_t+1　　x‘=x_t+1

where the Jacobian matrix of f is evaluated at the steady state. Adopting the notation introduced in Section 1.6, we can also derive a log-linearized system. Both systems are in the form of (2.11), e.g.,

$$\begin{bmatrix} f_{x'} & f_{y'} \end{bmatrix} \begin{bmatrix} E_t dx_{t+1} \\ E_t dy_{t+1} \end{bmatrix} = - \begin{bmatrix} f_x & f_y \end{bmatrix} \begin{bmatrix} dx_t \\ dy_t \end{bmatrix}. \qquad (2.28)$$

$$E_t f(y_{t+1}, y_t, x_{t+1}, x_t) = 0,$$

$$y_t = g(x_t, \sigma), \qquad x_{t+1} = h(x_t, \sigma) + \eta\sigma\varepsilon_{t+1},$$

We then define

$$\begin{aligned} 0 &= F(x, \sigma) \\ &\equiv Ef\left(g\left(h(x,\sigma) + \eta\sigma\varepsilon', \sigma\right), g(x,\sigma), h(x,\sigma) + \eta\sigma\varepsilon', x\right) \quad (2.30) \end{aligned}$$

=y_t+1 =y_t =x_t+1

We then use (2.30) to compute

$$\begin{aligned} F_\sigma(\bar{x}, 0) &= E\left\{f_{y'}\left[g_x\left(h_\sigma + \eta\varepsilon'\right) + g_\sigma\right] + f_y g_\sigma + f_{x'}\left(h_\sigma + \eta\varepsilon'\right)\right\} \\ &= f_{y'}\left[g_x h_\sigma + g_\sigma\right] + f_y g_\sigma + f_{x'} h_\sigma = 0, \end{aligned}$$

=dF/dσ

where all derivatives are evaluated at the non-stochastic steady state. This is a system of homogeneous linear equations for g_σ and h_σ. For a unique solution to exist, we must have

$$g_\sigma = h_\sigma = 0.$$

$$F_x(\bar{x}, 0) = f_{y'} g_x h_x + f_y g_x + f_{x'} h_x + f_x = 0.$$

=dF/dx

We can equivalently rewrite the above equation as

$$\begin{bmatrix} f_{x'} & f_{y'} \end{bmatrix} \begin{bmatrix} I \\ g_x \end{bmatrix} h_x = - \begin{bmatrix} f_x & f_y \end{bmatrix} \begin{bmatrix} I \\ g_x \end{bmatrix}. \tag{2.31}$$

Let $A = \begin{bmatrix} f_{x'} & f_{y'} \end{bmatrix}$, $B = - \begin{bmatrix} f_x & f_y \end{bmatrix}$, and $\hat{x}_t = x_t - \bar{x}$. Postmultiplying $\hat{x}_t$ on each side of (2.31) yields:

$$A \begin{bmatrix} I \\ g_x \end{bmatrix} h_x \hat{x}_t = B \begin{bmatrix} I \\ g_x \end{bmatrix} \hat{x}_t. \tag{2.32}$$

Using the fact that $\hat{x}_{t+1} = h_x \hat{x}_t$ and $\hat{y}_t = g_x \hat{x}_t$ for the linearized non-stochastic solution, we obtain

$$A \begin{bmatrix} \hat{x}_{t+1} \\ \hat{y}_{t+1} \end{bmatrix} = B \begin{bmatrix} \hat{x}_t \\ \hat{y}_t \end{bmatrix}. \tag{2.33}$$

1次近似(2.33)式は Blanchard-Kahnのモデルに変換できる

$$A \begin{bmatrix} \hat{x}_{t+1} \\ \hat{y}_{t+1} \end{bmatrix} = B \begin{bmatrix} \hat{x}_t \\ \hat{y}_t \end{bmatrix}. \tag{2.33}$$

Blanchard & Kahn (1980)

6.8.1 General Version

A linear model can be written (in what is known as a state space representation) as

State variables

$$B \begin{bmatrix} x_{t+1} \\ E_t y_{t+1} \end{bmatrix} = A \begin{bmatrix} x_t \\ y_t \end{bmatrix} + G\varepsilon_t, \tag{6.13}$$

Control variables

where x_t is an $(n \times 1)$ vector of predetermined variables at date t, y_t is an $(m \times 1)$ vector of non-predetermined variables at time t, $E_t y_{t+1}$ is the $(m \times 1)$ vector of expectations for the non-predetermined variables at date $t+1$, ε_t is a $(k \times 1)$ vector of stochastic shocks, A and B are $((n+m) \times (n+m))$ matrices, and G is an $((n+m) \times k)$ matrix. The difference between predetermined and non-predetermined variables is that the values of the predetermined variables at time $t+1$ do not depend on the values of the time $t+1$ shocks, while the values of the non-predetermined variables do depend on them. That is why, at

2nd order approximation

Our goal is to find an approximate solution for g and h. We shall consider local approximation around a particular point $(\bar{x}, \bar{\sigma})$. In applications, a convenient point to take is the non-stochastic steady state $(\bar{x}, \bar{\sigma})$. At this point, $f(\bar{y}, \bar{y}, \bar{x}, \bar{x}) = 0$. We then write the Taylor approximation up to the second order,

$$\begin{aligned} g(x,\sigma) &= g(\bar{x},0) + g_x(\bar{x},0)(x-\bar{x}) + g_\sigma(\bar{x},0)\sigma \\ &\quad + \frac{1}{2} g_{xx}(\bar{x},0)(x-\bar{x})^2 + g_{x\sigma}(\bar{x},0)(x-\bar{x})\sigma \\ &\quad + \frac{1}{2} g_{\sigma\sigma}(\bar{x},0)\sigma^2, \end{aligned}$$

$$\begin{aligned} h(x,\sigma) &= h(\bar{x},0) + h_x(\bar{x},0)(x-\bar{x}) + h_\sigma(\bar{x},0)\sigma \\ &\quad + \frac{1}{2} h_{xx}(\bar{x},0)(x-\bar{x})^2 + h_{x\sigma}(\bar{x},0)(x-\bar{x})\sigma \\ &\quad + \frac{1}{2} h_{\sigma\sigma}(\bar{x},0)\sigma^2, \end{aligned}$$

n_x

$n_x \times n_x$

$$E_t f(y_{t+1}, y_t, x_{t+1}, x_t) = 0,$$

$$y_t = g(x_t, \sigma), \qquad x_{t+1} = h(x_t, \sigma) + \eta\sigma\varepsilon_{t+1},$$

We then define

$$\begin{aligned} 0 &= F(x, \sigma) \\ &\equiv Ef\left(g\left(h(x,\sigma) + \eta\sigma\varepsilon', \sigma\right), g(x,\sigma), h(x,\sigma) + \eta\sigma\varepsilon', x\right) \quad (2.30) \end{aligned}$$

=y_t+1 =x_t+1

We compute

$$\begin{aligned} 0 = F_{xx}(\bar{x}, 0) = &[f_{y'y} g_x h_x + f_{y'y} g_x \\ &+ f_{y'x'} h_x + f_{y'x}] g_x h_x \quad n_x \\ &+ f_{y'} g_{xx} h_x h_x + f_{y'} g_x h_{xx} \quad n_x \times n_x \\ &+ [f_{yy'} g_x h_x + f_{yy} g_x + f_{yx'} h_x + f_{yx}] g_x + f_y g_{xx} \quad n_y \times n_x \\ &+ [f_{x'y'} g_x h_x + f_{x'y} g_x + f_{x'x'} h_x + f_{x'x}] h_x + f_{x'} h_{xx} \\ &\qquad n_x \qquad n_x \times n_x \\ &+ f_{xy'} g_x h_x + f_{xy} g_x + f_{xx'} h_x + f_{xx}. \end{aligned}$$

Since we know the derivatives of f as well as the first derivatives of g and h evaluated at $(\bar{y}, \bar{y}, x, x)$, it follows that the above equation represents a system of $n \times n_x \times n_x$ linear equations for the same number of unknowns given by the elements of g_{xx} and h_{xx}.

Similarly, we compute $g_{\sigma\sigma}$ and $h_{\sigma\sigma}$ by solving the equation $F_{\sigma\sigma}(\bar{x}, 0) = 0$. We compute

$$\begin{aligned} 0 \quad = \quad & F_{\sigma\sigma}(\bar{x}, 0) = f_{y'} g_x h_{\sigma\sigma} + f_{y'y'} g_x \eta g_x \eta I \\ & + f_{y'x'} \eta g_x \eta I + f_{y'} g_{xx} \eta \eta I + f_{y'} g_{\sigma\sigma} \\ & + f_y g_{\sigma\sigma} + f_{x'} h_{\sigma\sigma} \\ & + f_{x'y'} g_x \eta \eta I + f_{x'x'} \eta \eta I. \end{aligned}$$

This is a system of n linear equations for n unknowns $g_{\sigma\sigma}$ and $h_{\sigma\sigma}$.

Finally, we show that the cross derivatives $g_{\sigma x}$ and $h_{\sigma x}$ are equal to zero when evaluated at $(\bar{x}, 0)$. We compute

$$0 = F_{\sigma x}(\bar{x}, 0) = f_{y'} g_x h_{\sigma x} + f_{y'} g_{\sigma x} h_x + f_y g_{\sigma x} + f_{x'} h_{\sigma x}.$$

This is a system of $n \times n_x$ linear homogenous equations for $n \times n_x$ unknowns $g_{\sigma x}$ and $h_{\sigma x}$. If a unique solution exists, then the solution must be equal to zero.

Matlab codes of 2nd Approximation of policy function (Smitt-Grohe and Uribe, 2004, JEDC)

Matlab code

First-order approximation
gx_hx.m

Second-order approximation
gxx_hxx.m
gss_hss.m

Obtaining the derivatives of *f*
(requires Matlab's Symbolic Math Toolbox)
anal_deriv.m
num_eval.m

Model Simulation:
simu_2nd.m

Writing the Output of anal_deriv.m to an M-file: This program saves significant amount of computational time in applications in which the output of anal_deriv.m has to be evaluated repeatedly.
anal_deriv_print2f.m by Andrea Pescatori

Example 1: The neoclassical growth model:
Contents of Matlab Codes

- neoclassical_model_run.m → main program

- neoclassical_model.m →
  description of model equations (FOC and constraints)

- neoclassical_model_ss →
  set up of parameters
  and (non-stochastic) Steady State
- neoclassical_model_simul →
  calculation of IRF, Artificial data (Monte Carlo)
  and stochastic Steady State

Main code その1

```
26
27  %  analytical 2nd order derivatives of system: f
28  [fx,fxp,fy,fyp,fypyp,fypy,fypxp,fypx,fyyp,fyy,...
29      fyxp,fyx,fxpyp,fxpy,fxpxp,fxpx,fxyp,fxy,fxxp,fxx,f] = neoclassical_model;
30
31  %% Numerical Evaluation
32  % Non-stochastic  Steady State and Parameter Values
33  [SIG,DELTA,ALFA,BETTA,RHO,eta,c,cp,k,kp,a,ap,A,K,C]=neoclassical_model_ss;
34
35  %Order of approximation desired
36   approx = 2;
37
38  %Obtain numerical derivatives of f
39  num_eval
40
41  %First-order approximation of Policy function
42  [gx,hx] = gx_hx(nfy,nfx,nfyp,nfxp)
```

Main code その2

```
41  %First-order approximation of Policy function
42  [gx,hx] = gx_hx(nfy,nfx,nfyp,nfxp)
43
44  %Second-order approximation of Policy function
45  [gxx,hxx] = gxx_hxx(nfx,nfxp,nfy,nfyp,nfypyp,nfypy,nfypxp,...
46                      nfypx,nfyyp,nfyy,nfyxp,nfyx,nfxpyp,...
47                      nfxpy,nfxpxp,nfxpx,nfxyp,nfxy,nfxxp,nfxx,hx,gx)
48
49  [gss,hss] = gss_hss(nfx,nfxp,nfy,nfyp,nfypyp,nfypy,nfypxp,nfypx,...
50                      nfyyp,nfyy,nfyxp,nfyx,nfxpyp,nfxpy,nfxpxp,nfxpx,...
51                      nfxyp,nfxy,nfxxp,nfxx,hx,gx,gxx,eta)
52
53
54  %% Simulation of solution
55  neoclassical_model_simul;
56
```

Example 1: The neoclassical growth model

Sec 2 of Smitt-Grohe and Uribe (2004)

$$c_t^{-\gamma} = \beta E_t c_{t+1}^{-\gamma}[\alpha A_{t+1} k_{t+1}^{\alpha-1} + 1 - \delta],$$ Euler equation

$$c_t + k_{t+1} = A_t k_t^{\alpha} + (1-\delta)k_t,$$ Dynamics of capital

$$\ln A_{t+1} = \rho \ln A_t + \sigma \varepsilon_{t+1},$$ TFP shock process

control predetermined

for all $t \geqslant 0$, given k_0 and A_0. Let $y_t = c_t$ and $x_t = [k_t;\ \ln A_t]'$. Then

$$E_t f(y_{t+1}, y_t, x_{t+1}, x_t) = E_t \begin{bmatrix} y_{1t}^{-\gamma} - \beta y_{1t+1}^{-\gamma}[\alpha e^{x_{2t+1}} x_{1t+1}^{\alpha-1} + 1 - \delta] \\ y_{1t} + x_{1t+1} - e^{x_{2t}} x_{1t}^{\alpha} - (1-\delta)x_{1t} \\ x_{2t+1} - \rho x_{2t} \end{bmatrix},$$

Neoclassical_model.m

```
%(c) Stephanie Schmitt-Grohe and Martin Uribe
%Date July 17, 2001, revised 22-Oct-2004

%Define parameters
syms    SIG DELTA ALFA BETTA RHO

%Define variables
syms c cp k kp a ap

%Write equations fi, i=1:3
f1 = c + kp - (1-DELTA) * k - a * k^ALFA;
f2 = c^(-SIG) - BETTA * cp^(-SIG) * (ap * ALFA * kp^(ALFA-1) + 1 - DELTA);
f3 = log(ap) - RHO * log(a);

%Create function f
f = [f1;f2;f3];
```

define:

$$x_t = \begin{bmatrix} \ln k_t \\ \ln A_t \end{bmatrix}$$: Predetermined variables

and

$$y_t = \ln c_t.$$: Control variable

Then the non-stochastic steady-state values of y_t and x_t are, respectively:

$$\bar{y} = -0.8734.$$

and

$$\bar{x} = \begin{bmatrix} -1.7932 \\ 0 \end{bmatrix}.$$

```
29
30    % Define the vector
31    x = [k a];
32    y = c;
33    xp = [kp ap];
34    yp = cp;
35
```

```
function [fx,fxp,fy,fyp,fypyp,fypy,fypxp,fypx,fyyp,fyy,fyxp,fyx,fxpyp,fxpy,fxpxp,fxpx,fxyp,fxy,fxxp,fxx,f] = neoclassical_model
```

```
35
36    %Make f a function of the logarithm of the state and contr
37    f = subs(f, [x,y,xp,yp], (exp([x,y,xp,yp])));
38    %if line 36 gives an error (whichh it will for some versic
39    %f = subs(f, [x,y,xp,yp], transpose(exp([x,y,xp,yp])));
40
41    %Compute analytical derivatives of f
42     approx = 2;  %1:1st order, 2: 2nd order
43    [fx,fxp,fy,fyp,fypyp,fypy,fypxp,fypx,fyyp,...
44     fyy,fyxp,fyx,fxpyp,fxpy,fxpxp,fxpx,fxyp,...
45     fxy,fxxp,fxx]    =  anal_deriv(f,x,y,xp,yp,approx);
46
```

このページの内容は最新ではありません。最新版の英語を参照す

subs

シンボリック代入

構文

```
snew = subs(s,old,new)
snew = subs(s,new)
snew = subs(s)
```

```
sMnew = subs(sM,oldM,newM)
sMnew = subs(sM,newM)
sMnew = subs(sM)
```

例

単一置換

この式で a を 4 に置き換えます。

```
syms a b
subs(a + b,a,4)
```

ans = $b+4$

この式で a*b を 5 に置き換えます。

```
subs(a*b^2,a*b,5)
```

ans = $5b$

```
function [fx,fxp,fy,fyp,fypyp,fypy,fypxp,...
fypx,fyyp,fyy,fyxp,fyx,fxpyp,fxpy,fxpxp,fxpx,fxyp,fxy,fxxp,fxx]...
=anal_deriv(f,x,y,xp,yp,approx);
```

```
5   if nargin==5 %5つ目の入力が空欄のケース
6   approx=2;
7   end
8
9   nx = size(x,2);
10  ny = size(y,2);
11  nxp = size(xp,2);
12  nyp = size(yp,2);
13
14  n = size(f,1);
15
16  %Compute the first and second derivatives of f
17  fx = jacobian(f,x);
18  fxp = jacobian(f,xp);
19  fy = jacobian(f,y);
20  fyp = jacobian(f,yp);
21
```

```
22  if approx==2
23  fypyp = reshape(jacobian(fyp(:),yp),n,nyp,nyp);
24  fypy = reshape(jacobian(fyp(:),y),n,nyp,ny);
25  fypxp = reshape(jacobian(fyp(:),xp),n,nyp,nxp);
26  fypx = reshape(jacobian(fyp(:),x),n,nyp,nx);
27  fyyp = reshape(jacobian(fy(:),yp),n,ny,nyp);
28  fyy = reshape(jacobian(fy(:),y),n,ny,ny);
29  fyxp = reshape(jacobian(fy(:),xp),n,ny,nxp);
30  fyx = reshape(jacobian(fy(:),x),n,ny,nx);
31  fxpyp = reshape(jacobian(fxp(:),yp),n,nxp,nyp);
32  fxpy = reshape(jacobian(fxp(:),y),n,nxp,ny);
33  fxpxp = reshape(jacobian(fxp(:),xp),n,nxp,nxp);
34  fxpx = reshape(jacobian(fxp(:),x),n,nxp,nx);
35  fxyp = reshape(jacobian(fx(:),yp),n,nx,nyp);
36  fxy = reshape(jacobian(fx(:),y),n,nx,ny);
37  fxxp = reshape(jacobian(fx(:),xp),n,nx,nxp);
38  fxx = reshape(jacobian(fx(:),x),n,nx,nx);
39  else
40  fypyp=0; fypy=0; fypxp=0; fypx=0; fyyp=0; fyy=0;...
41      fyxp=0; fyx=0; fxpyp=0; fxpy=0; fxpxp=0;...
42      fxpx=0; fxyp=0; fxy=0; fxxp=0; fxx=0;
43  end
```

reshape

R2024b

既存の要素を再配列して配列を形状変更する

ページ内をすべて折りたたむ

構文

```
B = reshape(A,sz)
B = reshape(A,sz1,...,szN)
```

例

ベクトルを行列に形状変更

1 行 10 列のベクトルを 5 行 2 列の行列に形状変更します。

```
A = 1:10;
B = reshape(A,[5,2])
```

```
B = 5×2

     1     6
     2     7
     3     8
     4     9
     5    10
```

1次近似　gx_hx()

```
function [gx,hx,exitflag]=gx_hx(fy,fx,fyp,fxp,stake);

if nargin<5
    stake=1;
end
exitflag = 1;

%Create system matrices A,B
A = [-fxp -fyp];
B = [fx fy];
NK = size(fx,2);

%Complex Schur Decomposition
[s,t,q,z] = qz(A,B);

%Pick non-explosive (stable) eigenvalues
slt = (abs(diag(t))<stake*abs(diag(s)));
nk=sum(slt);

%Reorder the system with stable eigs in upper-left
[s,t,q,z] = ordqz(s,t,q,z,slt);

%Split up the results appropriately
z21 = z(nk+1:end,1:nk);
z11 = z(1:nk,1:nk);

s11 = s(1:nk,1:nk);
t11 = t(1:nk,1:nk);
```

```
%Identify cases with no/multiple solutions
if nk>NK
    warning('The Equilibrium is Locally Indeterminate')
    exitflag=2;
elseif nk<NK
    warning('No Local Equilibrium Exists')
    exitflag = 0;
end

if rank(z11)<nk;
    warning('Invertibility condition violated')
    exitflag = 3;
end

%Compute the Solution
z11i = z11\eye(nk);
gx = real(z21*z11i);
hx = real(z11*(s11\t11)*z11i);
```

2次近似

```
23
24  function [gxx,hxx] = gxx_hxx(fx,fxp,fy,fyp, ...
25      fypyp,fypy,fypxp,fypx,fyyp,fyy,fyxp, ...
26      fyx,fxpyp,fxpy,fxpxp,fxpx,fxyp,fxy,fxxp,fxx,hx,gx)
27
28  m=0;
29  nx = size(hx,1); %rows of hx and hxx
30  ny = size(gx,1); %rows of gx and gxx
31  n = nx + ny; %length of f
32  ngxx = nx^2*ny; %elements of gxx
33
34  sg = [ny nx nx]; %size of gxx
35  sh = [nx nx nx]; %size of hxx
36
37  Q = zeros(n*nx*(nx+1)/2,n*nx*nx);
38  q = zeros(n*nx*(nx+1)/2,1);
39  gxx=zeros(sg);
40  hxx=zeros(sh);
41  GXX=zeros(sg);
42  HXX=zeros(sh);
43
```

```
45  for i=1:n
46  for j=1:nx
47  %for k=1:nx
48  for k=1:j
49  m = m+1;
50
51  %First Term
52  q(m,1) = ( shiftdim(fypyp(i,:,:),1) * gx * hx(:,k) ...
53      + shiftdim(fypy(i,:,:),1) * gx(:,k) + shiftdim(fypxp(i,:,:),1) ...
54      *  hx(:,k) + shiftdim(fypx(i,:,k),1) )' * gx * hx(:,j);
55
56  % Second term
57  GXX(:) = kron(ones(nx^2,1),fyp(i,:)');
58  pGXX = permute(GXX,[2 3 1]);
59  pGXX(:) = pGXX(:) .* kron(ones(nx*ny,1),hx(:,j));
60  GXX=ipermute(pGXX,[2 3 1]);
61  pGXX = permute(GXX,[3 1 2]);
62  pGXX(:) = pGXX(:) .* kron(ones(nx*ny,1),hx(:,k));
63  GXX=ipermute(pGXX,[3 1 2]);
64  Q(m,1:ngxx)=GXX(:)';
65  GXX=0*GXX;
```

```
67  %Third term
68  HXX(:,j,k) = (fyp(i,:) * gx)';
69  Q(m,ngxx+1:end)=HXX(:)';
70  HXX = 0*HXX;
71
72  %Fourth Term
73  q(m,1) = q(m,1) + ( shiftdim(fyyp(i,:,:),1) * gx * hx(:,k) ...
74      +  shiftdim(fyy(i,:,:),1) * gx(:,k) + shiftdim(fyxp(i,:,:),1) ...
75      * hx(:,k) +  shiftdim(fyx(i,:,k),1) )' * gx(:,j);
76
77  %Fifth Term
78  GXX(:,j,k)=fy(i,:)';
79  Q(m,1:ngxx) = Q(m,1:ngxx) + GXX(:)';
80  GXX = 0*GXX;
81
82  %Sixth term
83  q(m,1) = q(m,1) + ( shiftdim(fxpyp(i,:,:),1) ...
84      * gx * hx(:,k) + shiftdim(fxpy(i,:,:),1) ...
85      * gx(:,k) + shiftdim(fxpxp(i,:,:),1) ...
86      * hx(:,k) + fxpx(i,:,k)')' * hx(:,j);
87
88  %Seventh Term
89  HXX(:,j,k)=fxp(i,:)';
90  Q(m,ngxx+1:end) = Q(m,ngxx+1:end) + HXX(:)';
91  HXX = 0*HXX;
92
```

```
92
93  %Eighth Term
94  q(m,1) = q(m,1) +  shiftdim(fxyp(i,j,:),1)
95  * gx * hx(:,k) +  shiftdim(fxy(i,j,:),1)
96  * gx(:,k) +  shiftdim(fxxp(i,j,:),1) * hx(:,k) + fxx(i,j,k);
97
98  end %k
99  end %j
100 end %i
101
102 A = temp(nx,ny);
103
104 Qt = Q* A;
105
106 xt = -Qt\q;
107 x = A* xt;
108
109 gxx(:)=x(1:ngxx);
110 hxx(:) = x(ngxx+1:end);
111
```

The coefficients of the linear terms are:

$$g_x = [0.2525 \quad 0.8417]$$

and

$$h_x = \begin{bmatrix} 0.4191 & 1.3970 \\ 0.0000 & 0.0000 \end{bmatrix}.$$

The coefficients of the quadratic terms are given by:

$$g_{xx}(:,:,1) = [-0.0051 \quad -0.0171],$$

$$g_{xx}(:,:,2) = [-0.0171 \quad -0.0569]$$

and

$$h_{xx}(:,:,1) = \begin{bmatrix} -0.0070 & -0.0233 \\ 0 & 0 \end{bmatrix},$$

$$h_{xx}(:,:,2) = \begin{bmatrix} -0.0233 & -0.0778 \\ 0 & 0 \end{bmatrix}.$$

Finally, the coefficients of the quadratic terms in σ are:

$$g_{\sigma\sigma} = -0.1921$$

and

$$h_{\sigma\sigma} = \begin{bmatrix} 0.4820 \\ 0 \end{bmatrix}.$$

$$\begin{aligned} g(x,\sigma) = {} & g(\bar{x},0) + g_x(\bar{x},0)(x-\bar{x}) + g_\sigma(\bar{x},0)\sigma \\ & + \frac{1}{2}g_{xx}(\bar{x},0)(x-\bar{x})^2 + g_{x\sigma}(\bar{x},0)(x-\bar{x})\sigma \\ & + \frac{1}{2}g_{\sigma\sigma}(\bar{x},0)\sigma^2, \end{aligned}$$

Matlabの計算結果

```
gx =

    0.2525    0.8417


hx =

    0.4191    1.3970
         0         0


gxx(:,:,1) =

   -0.0051   -0.0171


gxx(:,:,2) =

   -0.0171   -0.0569
```

```
hxx(:,:,1) =

   -0.0070   -0.0233
         0         0


hxx(:,:,2) =

   -0.0233   -0.0778
         0         0


gss =

   -0.1921


hss =

    0.4820
         0
```

Policy function

$$y_t = g(x_t, \sigma),$$

Then, the laws of motion of these two variables are given by

$$\hat{c}_t = 0.2525\hat{k}_t + 0.8417\hat{A}_t + \frac{1}{2}[-0.0051\hat{k}_t^2 - 0.0341\hat{k}_t\hat{A}_t - 0.0569\hat{A}_t^2 - 0.1921\sigma^2]$$

and

$$\hat{k}_{t+1} = 0.4191\hat{k}_t + 1.3970\hat{A}_t$$

$$x_{t+1} = h(x_t, \sigma) + \eta\sigma\varepsilon_{t+1},$$

$$+\frac{1}{2}\left[-0.0070\hat{k}_t^2 - 0.0467\hat{k}_t\hat{A}_t - 0.0778\hat{A}_t^2 + 0.4820\sigma^2\right].$$

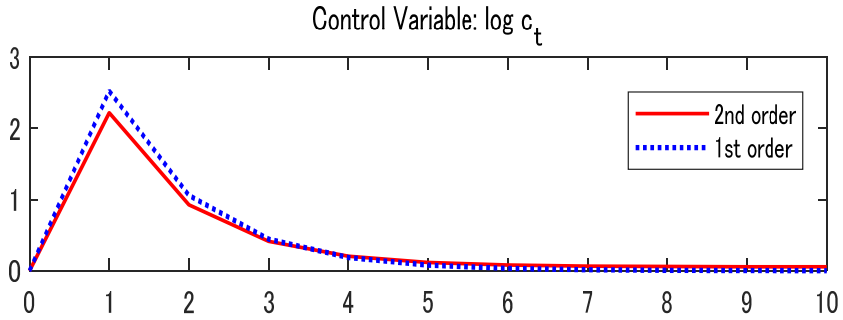
It can be verified that these numbers coincide with those obtained by Sims (2000).

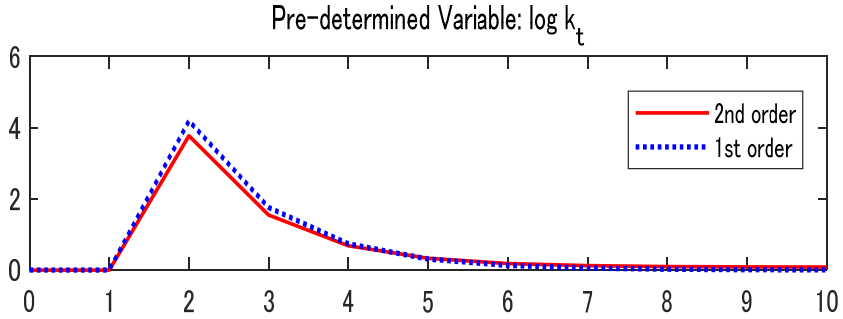
Neoclassical_simul.m

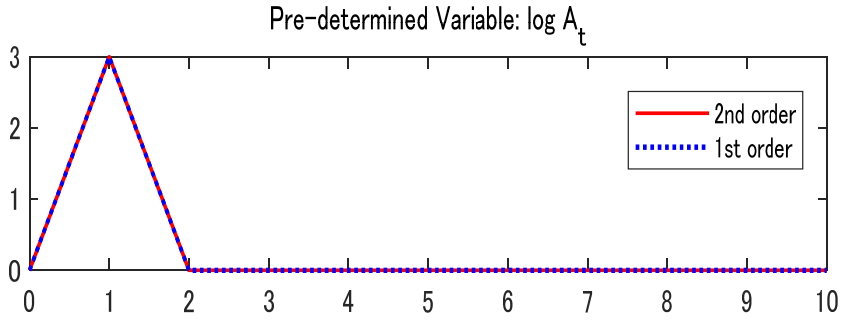
```
%% Check Solution
% cal   unconditional_mean
sig = 1;  % St Dev of shock
[Ey,Ex] = unconditional_mean(gx, hx, gxx, hxx, gss, hss, eta, sig)

x0 = Ex;
e = [ 0 0; zeros(200,2)];
[Y02,X02] = simu_2nd(gx, hx, gxx, hxx, gss, hss, eta, sig, x0, e);
x_ss1 = X02(end,:)' % stochastic steady state

% cal stochastic steady state by fixed point
x_d = zeros(4,1);  % x_d = zeros(4*2,1);
 x0_ss = csolve('eq_stochastic_steady_state',x_d ,[],1e-5,2000,hx, hxx, hss, sig )
%   x0_ss2   = x0_ss(1:4,1)+x0_ss(5:8,1)
[Error] = eq_stochastic_steady_state(x0_ss , hx, hxx, hss, sig )

```

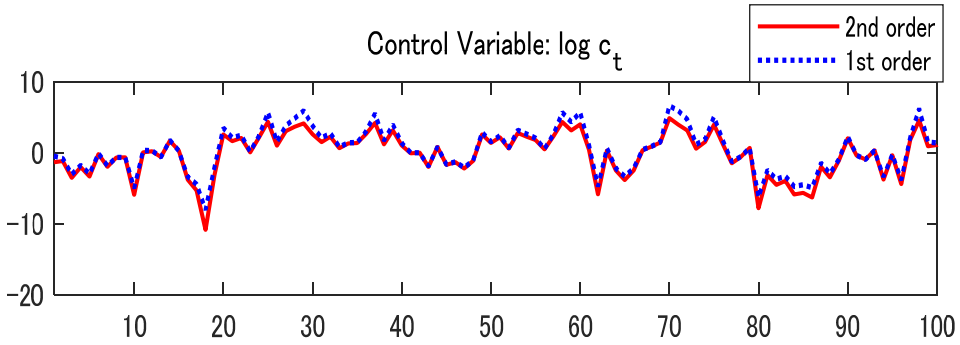
```
% Impulse Response Function
% sig = 0.1;           % St Dev of shock
x0_1 = [0;0;0;0];        % initial values of predetermined variables
x0_2 = x_ss1;
h = 40;         % horizon of IRF
e = [ 1 0; zeros(h,2)]; % generations of shocks
t = 0:1:h+1;

[Y2,X2] = simu_2nd(gx, hx, gxx, hxx, gss, hss, eta, sig, x0_2, e);
[Y1,X1] = simu_1st(gx, hx, eta, sig, x0_1, e);

figure('Name','IRF of Kim Model')
subplot(3,1,1)
     plot(t,Y2-Y2(1,1),'r-','LineWidth',1.5);
   hold on
     plot(t,Y1-Y1(1,1),'b:','LineWidth',1.5);
   hold off
     title('Control Variable: log c_t')
     legend('2nd order', '1st order')
     xlim([0 h])
%      ylim( [-0.025 0.025])
```

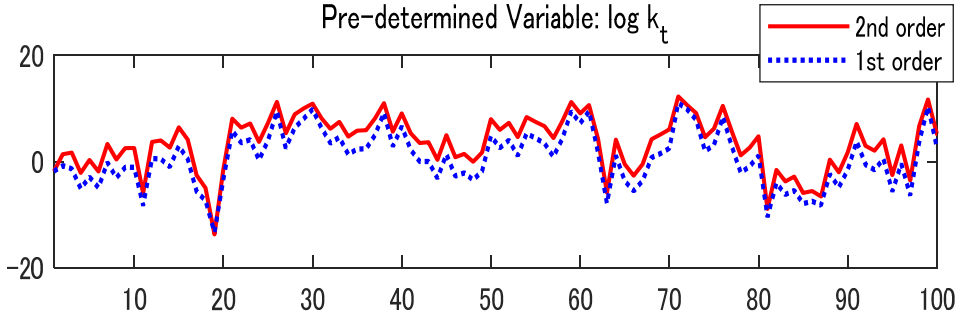
インパルス応答の1次近似と2次近似の比較

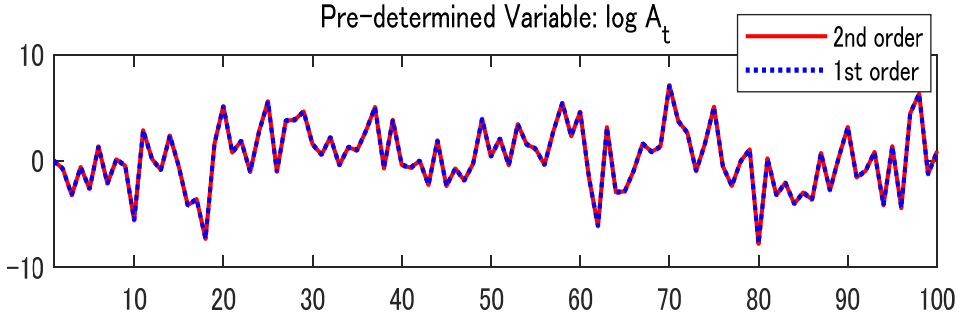












Example 2: A two-country neoclassical model:
Contents of Matlab Codes

- kim_model_run.m → main program
- kim_model.m →
  description of model equations (FOC and constraints)
- kim_model_ss →
  set up of parameters
  and (non-stochastic) Steady State
- kim_model_simul →
  calculation of IRF, Artificial data (Monte Carlo)
  and stochastic Steady State

Example 2: A two-country neoclassical model with complete asset markets

The planner's objective function is given by

$$E_0 \sum_{t=0}^{\infty} \beta^t \left[\frac{C_{1t}^{1-\gamma} - 1}{1-\gamma} + \frac{C_{2t}^{1-\gamma} - 1}{1-\gamma} \right],$$

where C_{it}, $i = 1, 2$, denotes consumption of the representative household of country i in period t. The planner maximizes this utility function subject to the budget constraint

$$C_{1t} + C_{2t} + K_{1t+1} - (1-\delta)K_{1t} + K_{2t+1} - (1-\delta)K_{2t} = A_{1t}K_{1t}^{\alpha} + A_{2t}K_{2t}^{\alpha},$$

where K_{it} denotes the stock of physical capital in country i and A_{it} is an exogenous technology shock whose law of motion is given by

$$\ln A_{it} = \rho_i \ln A_{it-1} + \sigma \varepsilon_{it}, \qquad i = 1, 2,$$

where $\varepsilon_{it} \sim NIID(0, 1)$ and $\rho_i \in (-1, 1)$. The optimality conditions associated with this problem are the above period-by-period budget constraint and

$$C_{1t} = C_{2t},$$

$$C_{1t}^{-\gamma} = \beta E_t C_{1t+1}^{-\gamma} [\alpha A_{1t+1} K_{1t+1}^{\alpha-1} + (1-\delta)],$$

$$C_{1t}^{-\gamma} = \beta E_t C_{1t+1}^{-\gamma} [\alpha A_{2t+1} K_{2t+1}^{\alpha-1} + (1-\delta)].$$

kim_model.m

```
%Define parameters
syms GAMA DELTA ALFA RHO BETTA

%Define variables
syms c cp k1 k1p k2 k2p a1 a1p a2 a2p
%Write equations (e1 e2 e3 e4 e5)
e1 = c^(-GAMA) - BETTA * cp^(-GAMA) * (ALFA * a1p * k1p^(ALFA-1) + 1 - DELTA);

e2 = c^(-GAMA) - BETTA * cp^(-GAMA) * (ALFA * a2p * k2p^(ALFA-1) + 1 - DELTA);

e3 = 2 * c + k1p - (1-DELTA) * k1 + k2p - (1-DELTA) * k2 - a1 * k1^(ALFA) - a2 * k2^(ALFA);

e4 = log(a1p) - RHO * log(a1);

e5 = log(a2p) - RHO * log(a2);

%Create function f
f = [e1;e2;e3;e4;e5];
```

We use the following parameter values: $\gamma = 2$; $\delta = 0.1$; $\alpha = 0.3$; $\rho = 0$; and $\beta = 0.95$.

Given this parameterization, the second-order approximation to the policy function is given by:

$$\hat{K}_{1t+1} = [\,0.4440 \quad 0.4440 \quad 0.2146 \quad 0.2146\,] \begin{bmatrix} \hat{K}_{1t} \\ \hat{K}_{2t} \\ \hat{A}_{1t} \\ \hat{A}_{2t} \end{bmatrix}$$

$$+ \frac{1}{2}[\,\hat{K}_{1t} \quad \hat{K}_{2t} \quad \hat{A}_{1t} \quad \hat{A}_{2t}\,]$$

$$\times \begin{bmatrix} 0.22 & -0.18 & -0.023 & -0.088 \\ -0.18 & 0.22 & -0.088 & -0.023 \\ -0.023 & -0.088 & 0.17 & -0.042 \\ -0.088 & -0.023 & -0.042 & 0.17 \end{bmatrix} \begin{bmatrix} \hat{K}_{1t} \\ \hat{K}_{2t} \\ \hat{A}_{1t} \\ \hat{A}_{2t} \end{bmatrix}$$

$$- \frac{1}{2} 0.166\sigma^2,$$

$$\hat{K}_{2t+1} = \hat{K}_{1t+1},$$

$$[h(x,\sigma)]^j = [h(\bar{x},0)]^j + [h_x(\bar{x},0)]^j_a [(x-\bar{x})]_a + [h_\sigma(\bar{x},0)]^j [\sigma]$$

$$+ \frac{1}{2}[h_{xx}(\bar{x},0)]^j_{ab}[(x-\bar{x})]_a[(x-\bar{x})]_b$$

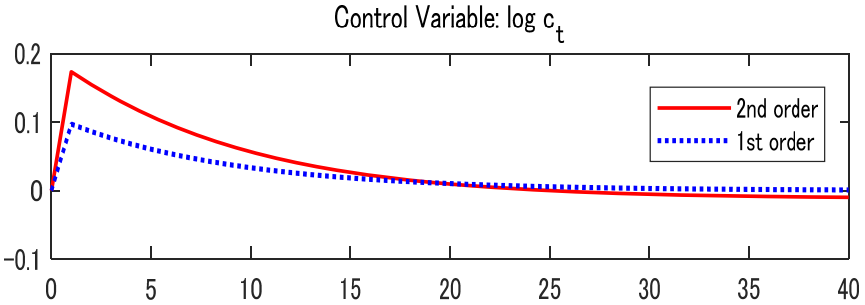
$$+ \frac{1}{2}[h_{x\sigma}(\bar{x},0)]^j_a[(x-\bar{x})]_a[\sigma]$$

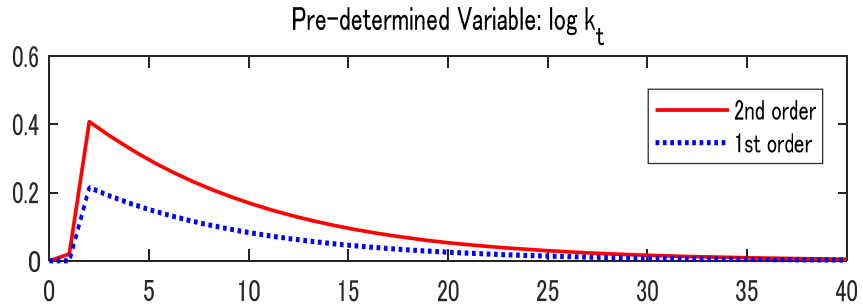
$$+ \frac{1}{2}[h_{\sigma x}(\bar{x},0)]^j_a[(x-\bar{x})]_a[\sigma]$$

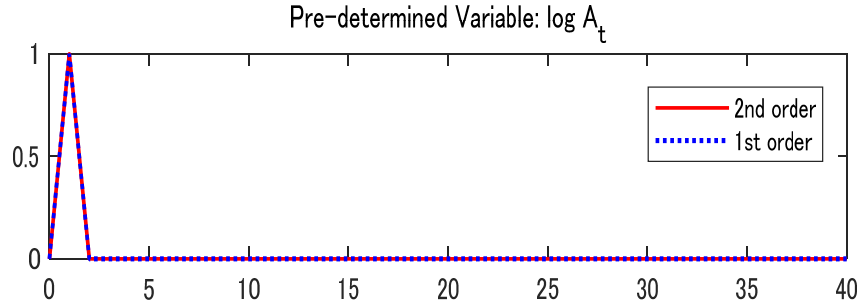
$$+ \frac{1}{2}[h_{\sigma\sigma}(\bar{x},0)]^j[\sigma][\sigma],$$

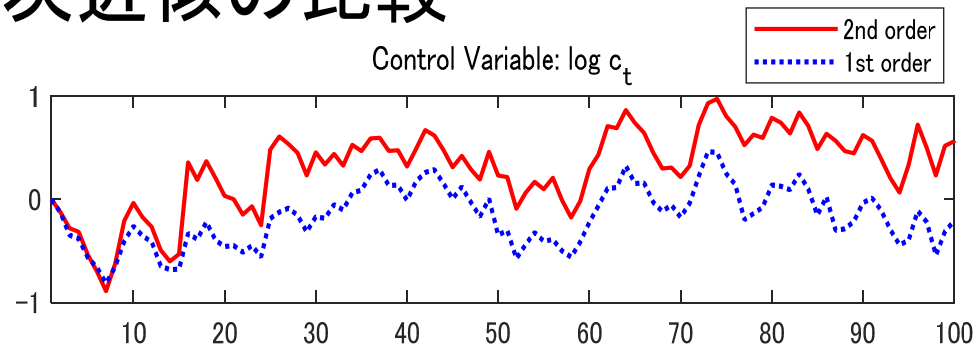
```
gx =

    0.2013    0.2013    0.0973    0.0973


hx =

    0.4440    0.4440    0.2146    0.2146
    0.4440    0.4440    0.2146    0.2146
         0         0         0         0
         0         0         0         0


gxx(:,:,1) =

    0.1013   -0.0796   -0.0093   -0.0385


gxx(:,:,2) =

   -0.0796    0.1013   -0.0385   -0.0093


gxx(:,:,3) =

   -0.0093   -0.0385    0.0787   -0.0186


gxx(:,:,4) =

   -0.0385   -0.0093   -0.0186    0.0787


hxx(:,:,1) =

    0.2178   -0.1812   -0.0232   -0.0876
    0.2178   -0.1812   -0.0232   -0.0876
         0         0         0         0
         0         0         0         0


hxx(:,:,2) =

   -0.1812    0.2178   -0.0876   -0.0232
   -0.1812    0.2178   -0.0876   -0.0232
         0         0         0         0
         0         0         0         0
```

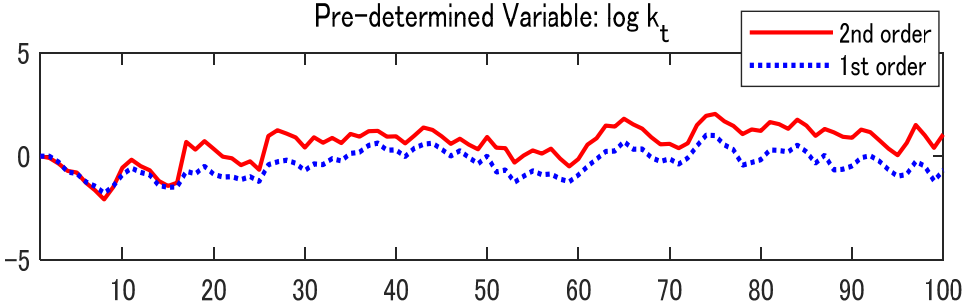
インパルス応答の1次近似と2次近似の比較

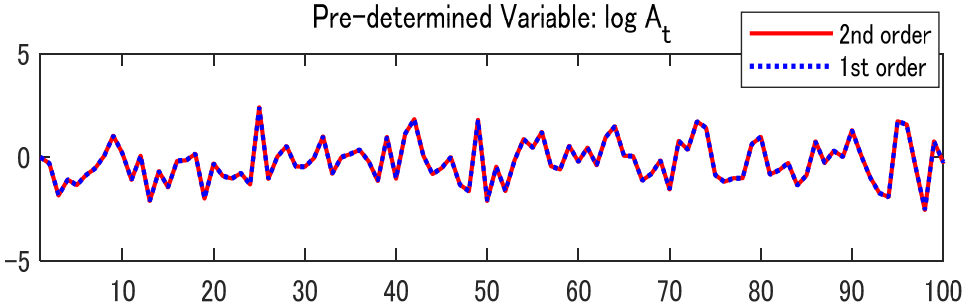












Example 3: An asset pricing model:
Contents of Matlab Codes

- asset_run.m → main program

- asset_model.m →
  description of model equations (FOC and constraints)

- asset_ss →
  set up of parameters
  and (non-stochastic) Steady State

- asset_model_simul →
  calculation of IRF, Artificial data (Monte Carlo)
  and stochastic Steady State

Example 3: An asset pricing model

Consider the following endowment economy analyzed by Burnside (1998) and Collard and Juillard (2001b). The representative agent maximizes the lifetime utility function

$$E_0 \sum_{t=0}^{\infty} \beta^t \frac{C_t^{\theta}}{\theta},$$

subject to

$$p_t e_{t+1} + C_t = p_t e_t + d_t e_t$$

and a borrowing limit that prevents agents from engaging in Ponzi games. In the above expressions, C_t denotes consumption, p_t the relative price of trees in terms of consumption goods, e_t the number of trees owned by the representative household at the beginning of period t, and d_t the dividends per tree in period t. Dividends are

assumed to follow an exogenous stochastic process given by

$$d_{t+1} = e^{x_{t+1}} d_t,$$

where e^{x_t} denotes the gross growth rate of dividends. The natural logarithm of the gross growth rate of dividends, x_t, is assumed to follow an exogenous AR(1) process

$$x_{t+1} = (1 - \rho)\bar{x} + \rho x_t + \sigma\eta\varepsilon_{t+1},$$

and $\varepsilon_t \sim NIID(0, 1)$.[14]

The optimality conditions associated with the household's problem are the above budget constraint, the borrowing limit, and

$$p_t C_t^{\theta-1} = \beta E_t C_{t+1}^{\theta-1}(p_{t+1} + d_{t+1}).$$

Burnside (1998) shows that the non-explosive solution to this equation is of the form

$$y_t \equiv g(x_t, \sigma) = \sum_{i=1}^{\infty} \beta^i \mathrm{e}^{a_i + b_i(x_t - \bar{x})}, \tag{9}$$

where

$$a_i = \theta \bar{x} \mathrm{i} + \frac{\theta^2 \sigma^2 \eta^2}{2(1-\rho)^2} \left[\mathrm{i} - \frac{2\rho(1-\rho^i)}{1-\rho} + \frac{\rho^2(1-\rho^{2i})}{1-\rho^2} \right]$$

and

$$b_i = \frac{\theta \rho (1-\rho^i)}{1-\rho}.$$

Asset_model.m

```
13    %Define parameters
14    syms XBAR RHO THETA BETTA
15
16    %Define variables
17    syms p pp a ap
18
19    %Write equations (e1 and e2)
20    e1 = -p + BETTA * (1 + pp) * exp(THETA * ap);
21
22    e2 = -ap + (1-RHO) * XBAR + RHO * a;
23
24    %Create function f
25    f = [e1;e2];
26
```

A second-order approximation of (9) around $x_t = \bar{x}$ and $\sigma = 0$ yields

$$y_t \approx g(\bar{x}, 0) + g_x(x_t - \bar{x}) + \frac{1}{2} g_{xx}(x_t - \bar{x})^2 + \frac{1}{2} g_{\sigma\sigma}\sigma^2, \tag{10}$$

where

$$g_x = \frac{\theta\rho\beta e^{\theta\bar{x}}}{(1 - \beta e^{\theta\bar{x}})(1 - \beta e^{\theta\bar{x}}\rho)},$$

$$g_{xx} = \left(\frac{\rho\theta}{1-\rho}\right)^2 \left[\frac{\beta e^{\theta\bar{x}}}{1 - \beta e^{\theta\bar{x}}} - \frac{2\beta e^{\theta\bar{x}}\rho}{1 - \beta e^{\theta\bar{x}}\rho} + \frac{\beta e^{\theta\bar{x}}\rho^2}{1 - \beta e^{\theta\bar{x}}\rho^2}\right],$$

and

$$g_{\sigma\sigma} = \left(\frac{\theta\eta}{1-\rho}\right)^2 \left[\frac{\beta e^{\theta\bar{x}}}{(1 - \beta e^{\theta\bar{x}})^2} + \left(\frac{\rho^2}{1-\rho^2} - \frac{2\rho}{1-\rho}\right) \times \frac{\beta e^{\theta\bar{x}}}{1 - \beta e^{\theta\bar{x}}} + \frac{2\rho^2}{1-\rho}\frac{\beta e^{\theta\bar{x}}}{1 - \beta e^{\theta\bar{x}}\rho} - \frac{\rho^4}{1-\rho^2}\frac{\beta e^{\theta\bar{x}}}{1 - \beta e^{\theta\bar{x}}\rho^2}\right].$$

We follow Burnside (1998) and Collard and Juillard (2001b) and use the calibration $\beta = 0.95$, $\theta = -1.5$, $\rho = -0.139$, $\bar{x} = 0.0179$, and $\eta = 0.0348$. Then, evaluating the above expressions we obtain

$$y_t \approx 12.30 + 2.27(x_t - \bar{x}) + \frac{1}{2} 0.42(x_t - \bar{x})^2 + \frac{1}{2} 0.35\sigma^2.$$

```
>> main_asset_model
gx =
    2.2731

hx =
   -0.1390

gxx =
    0.4205

hxx =
     0

gss =
    0.3507

hss =
     0

Ey =
    0.1756

Ex =
     0

x0_ss =
     0
```

インパルス応答の1次近似と2次近似の比較